

Relatório exploratório - Expressões, Atribuição e Estruturas de Controle

1. Começando a explorar o material e entendendo a precedência

Quando comecei a olhar todo o material, percebi que a proposta da aula é bem prática. Em vez de só apresentar regras, vários exemplos pedem para eu tentar imaginar o que o código vai mostrar antes de executar. Eu gostei dessa ideia porque algumas respostas parecem óbvias à primeira vista, mas mudam totalmente quando eu lembro das regras de cada linguagem. Ao longo dos arquivos aparecem C, Java, Python, JavaScript, Go, Rust, PHP, Ruby e Lisp, e isso deixa bem claro que um mesmo problema pode ser tratado de jeitos diferentes.

O primeiro assunto que explorei foi precedência e associatividade. Pra mim, a principal ideia foi entender que o computador não executa uma expressão simplesmente da esquerda para a direita. Os operadores têm prioridades. Um exemplo que me pegou foi o Python com `-2 ** 2`. Minha primeira vontade seria pensar em $(-2)^2$ e responder 4, mas o resultado é -4, porque a potência é feita antes do sinal de menos. Quando eu coloco os parênteses, aí sim `(-2) ** 2` vira 4.

Outro exemplo importante é `2 ** 3 ** 2`. Em Python e JavaScript, a potência associa à direita, então a conta vira `2 ** (3 ** 2)`, resultando em 512. Isso mostra como uma expressão pequena pode mudar bastante só por causa da associatividade. Também achei interessante o caso do Java com `1 + 2 + "3" + 4 + 5`. Primeiro `1 + 2` dá 3, mas depois aparece uma String e o operador `+` passa a concatenar. O resultado final fica 3345.

A divisão também trouxe uma pegadinha que eu considero muito importante. Em Java, `7 / 2` usa divisão inteira e resulta em 3. Se depois eu multiplicar por 2.0, o valor vira 6.0, porque a parte decimal já foi perdida antes. Pra mim, a maior lição dessa primeira parte foi simples: quando uma expressão mistura muitos operadores ou tipos, usar parênteses e deixar a intenção bem clara evita bastante confusão.

2. Efeitos colaterais e sobrecarga de operadores

Depois da precedência, fui para a parte de efeitos colaterais. O exemplo principal usa uma variável global chamada `a`, que começa com 5, e uma função que muda esse valor para 17 e retorna 3. A expressão `a = a + fun1()` parece simples, mas o resultado depende da ordem em que a linguagem avalia as partes. Em Java, a avaliação é da esquerda para a direita, então o valor 5 é lido antes da função alterar a variável e o resultado fica 8. Em C, a situação é mais complicada porque a ordem desse tipo de avaliação não é garantida da mesma forma.

Isso me fez perceber que o problema não é só uma função alterar alguma coisa. O problema aparece quando essa alteração acontece no meio de uma expressão e eu começo a depender de uma ordem que talvez nem esteja definida. Eu posso até conseguir um resultado específico no meu compilador, mas isso não quer dizer que o código seja uma boa ideia. Pra mim, o jeito mais seguro é separar as etapas e não colocar várias mudanças de estado dentro da mesma expressão.

Na sequência, explorei a sobrecarga de operadores. O `+` foi o exemplo mais fácil de entender, porque em várias linguagens ele soma números e também junta textos. O material mostra que isso pode ser útil, mas também pode confundir quando o significado muda no meio da expressão. Em Java, por exemplo, `'a' + 1` vira 98 porque o caractere é tratado como valor numérico, enquanto `"" + 'a' + 1` vira `a1` porque a String faz o `+` funcionar como concatenação.

O Python mostra um lado interessante da sobrecarga criada pelo próprio programador. Uma classe `Vetor` pode definir `__add__` e permitir que eu escreva `v1 + v2` de um jeito bem natural. Rust também permite definir comportamento para operadores usando traits. Já Java não deixa criar esse tipo de sobrecarga para classes próprias, então tipos como `BigDecimal` usam métodos como `add` e `multiply`. No fim, eu entendi que a sobrecarga pode deixar o código bonito e fácil de ler quando faz sentido, mas também pode esconder comportamentos que eu não esperava.

3. Conversões, coerções e as armadilhas dos tipos

A parte de conversões foi uma das que mais me chamou atenção porque tem várias situações em que o programa aceita o código, mas o resultado não fica exatamente como eu imaginava. O material separa bem a ideia de alargamento, estreitamento, cast e coerção. No alargamento, eu passo um valor para um tipo que suporta uma faixa maior. Mesmo assim, isso não significa que nunca exista perda de precisão. O exemplo do Java mostra um int grande sendo convertido para float e voltando com outro valor.

No estreitamento, o risco fica ainda mais claro. Converter 3.99 para int gera 3, porque a parte decimal é descartada. O exemplo de (byte) 200 também mostra como uma conversão pode produzir um valor totalmente diferente do que uma pessoa iniciante esperaria. Eu achei isso importante porque deixa claro que cast não é uma forma de 'consertar' qualquer incompatibilidade; às vezes ele simplesmente força a conversão e aceita a perda de informação.

Go e Rust aparecem como linguagens mais exigentes. Elas pedem conversões explícitas em vários casos em que C ou Java fazem ajustes automáticos. No começo isso pode parecer mais chato, mas eu percebi que essa obrigação deixa a intenção mais visível e diminui alguns erros silenciosos.

JavaScript e PHP foram os exemplos mais curiosos. Em JavaScript, "5" - 2 resulta em 3, mas "5" + 2 resulta em "52". O mesmo símbolo + pode puxar a operação para concatenação. Também aparecem comparações como 0 == "", 0 == "0" e "" == "0", com resultados que não seguem uma lógica intuitiva. Por isso o material reforça o uso de === e !==. Em PHP, o exemplo dos textos começando com 0e mostra como a comparação com coerção pode até virar um problema de segurança em situações de autenticação.

O que eu levei dessa parte foi que conversão automática facilita a escrita, mas pode esconder decisões. Quando a linguagem me obriga a converter de forma explícita, eu penso mais no que estou fazendo.

4. Booleanos e avaliação em curto-circuito

Quando cheguei nas expressões booleanas, vi de novo como o mesmo código pode ter significados diferentes entre linguagens. Um exemplo muito bom é $3 > 2 > 1$. Em C, primeiro $3 > 2$ vira 1 e depois a linguagem compara $1 > 1$, que é falso. Em Python, a expressão é entendida como uma comparação encadeada, equivalente a $(3 > 2)$ and $(2 > 1)$, e o resultado é verdadeiro. É um detalhe pequeno, mas que mostra como eu não posso levar automaticamente a regra de uma linguagem para outra.

Também aparece a ideia de valores que contam como verdadeiros ou falsos. Em Python, 0, String vazia, lista vazia e None contam como falso. Já "0" e [0] contam como verdadeiro. Em C, qualquer número diferente de zero é tratado como verdadeiro. Em Java, Go e Rust, o if exige um valor booleano de verdade. Eu gostei dessa comparação porque dá para perceber a diferença entre linguagens mais flexíveis e linguagens que preferem barrar o erro no compilador.

A avaliação em curto-circuito foi outra parte que achei muito útil. Em Java, a expressão `s != null && s.length() > 0` é segura porque, se s for null, a segunda parte nem é executada. Se eu trocar `&&` por `&`, as duas partes são avaliadas e aparece uma `NullPointerException`. Ou seja, curto-circuito não é só uma questão de desempenho; ele também pode proteger acessos que só fazem sentido quando a primeira condição foi satisfeita.

No JavaScript, o material liga esse assunto aos operadores `?.` e `??`. O `?.` permite acessar algo dentro de um objeto sem quebrar o programa quando aparece null ou undefined. O `??` é útil quando zero é um valor válido. Se eu usar quantidade `|| 10` e quantidade for 0, o resultado vira 10. Com quantidade `?? 10`, o zero é mantido. Pra mim, essa parte mostrou que condições pequenas podem carregar bastante comportamento escondido e que vale a pena saber exatamente o que cada operador faz.

5. Atribuição e modo misto

Na parte de atribuição, comecei a perceber que até o simples sinal de igual muda bastante de uma linguagem para outra. Em C, a atribuição é uma expressão, então ela devolve um valor. Isso permite fazer coisas como `sum = count = 0` ou colocar uma atribuição dentro de um `while`. É uma característica útil, mas também abre espaço para o clássico erro de escrever `if (x = y)` quando a intenção era comparar com `==`.

Java e C# também tratam a atribuição como expressão, mas o `if` exige booleano, então vários erros são barrados. Python segue outra linha: a atribuição normal não é uma expressão. Eu não posso simplesmente usar `n = 10` dentro de um `if`. Mesmo assim, existe o operador morsa `:=` para casos específicos em que faz sentido atribuir e testar ao mesmo tempo. Go também separa bem essas ideias e nem o incremento `x++` pode ser usado como parte de outra expressão.

Rust me chamou atenção pela imutabilidade padrão. Quando eu escrevo `let x = 5`, a variável não muda livremente como acontece em C ou Java. Se eu usar `let x = x + 1`, na verdade estou criando uma nova variável com o mesmo nome. Esse sombreamento aparece em outras partes do material e é uma ideia que eu precisei observar com cuidado para não confundir com uma simples reatribuição.

O modo misto também aparece nessa parte. Java aceita alguns alargamentos automaticamente, mas não aceita estreitamentos sem cast. C é mais permissivo e pode transformar um `double` em `int` silenciosamente. Uma pegadinha que eu considero muito importante é o cálculo `double taxa = acertos / total * 100`. Se `acertos` e `total` forem inteiros, `7 / 10` resulta em 0 antes de chegar no `double`. Depois disso, não adianta a variável final ser real: o valor já foi perdido. Para corrigir, eu posso usar `100.0`, fazer cast ou garantir que a divisão já aconteça com número real. Essa parte me mostrou que o tipo dos operandos importa tanto quanto o tipo da variável que recebe o resultado.

6. Seleção: if, else, switch e match

Na parte de seleção, o exemplo do else pendente foi um dos mais fáceis de visualizar. Em C, Java e outras linguagens parecidas, o else pertence ao if mais próximo que ainda não tem else, mesmo que a indentação pareça indicar outra coisa. Isso quer dizer que eu posso olhar o código, achar que entendi a estrutura e estar errada porque o compilador ignora a organização visual dos espaços. Por isso o material reforça tanto o uso de chaves.

Essa ideia se conecta diretamente com o caso real do goto fail, que aparece depois. Um bloco sem chaves deixa muito mais fácil uma linha parecer estar dentro de um if quando, na prática, está fora. Eu entendi que usar chaves mesmo quando existe apenas uma linha não é frescura: é uma forma de deixar o código menos ambíguo para quem lê e mais seguro para futuras alterações.

No switch de C, aparece outra característica que pode causar erro: a queda de um case para os seguintes quando não existe break. O exemplo mostra que escolher um caso pode acabar executando vários outros e até chegar ao default. Em Java tradicional acontece algo parecido. Já o Java moderno oferece switch com setas, o que evita essa queda automática. Em Go, cada case termina sozinho, e cair para o próximo exige fallthrough de propósito.

Rust vai ainda mais longe com o match. O compilador exige que todos os casos possíveis sejam cobertos. Se eu esquecer uma possibilidade, o código nem compila. Eu achei isso interessante porque a linguagem transforma uma preocupação do programador em uma regra que o compilador consegue verificar.

Python 3.10 também aparece com match, mas com uma ideia mais ampla de padrões. Ele consegue reconhecer formatos de listas e separar partes do valor. No fim dessa parte, eu fiquei com a impressão de que as linguagens mais recentes tentam tornar a seleção menos sujeita a ambiguidades. Chaves obrigatórias, ausência de queda automática e verificação de todos os casos são escolhas que deixam o código um pouco mais rígido, mas também mais previsível.

7. Laços, iteradores e o caso do Zune

A parte de iteração mostrou diferenças bem claras entre linguagens. Em C, o `for` é extremamente flexível. Eu posso mudar a variável de controle dentro do corpo do laço e isso afeta as próximas repetições. No exemplo do material, `i` também recebe `i += 2` dentro do bloco, além do incremento normal do `for`, e por isso a sequência exibida pula de 0 para 3, depois 6 e 9.

Em Python, o `for` funciona de outro jeito. Ele percorre um iterável. Se eu fizer `for i in range(0, 10)` e mudar `i` dentro do corpo, isso não altera os próximos valores entregues pelo `range`. O material também mostra `range` com passo negativo e o uso de `enumerate`. Pra mim, essa comparação foi ótima porque deixa claro que dois laços chamados de `for` podem ter modelos bem diferentes por trás.

Java e Rust mostram rótulos para sair de laços aninhados. Isso é útil quando uma condição encontrada dentro de um laço interno precisa encerrar também o laço externo. JavaScript traz outra pegadinha muito prática: `for...in` percorre as chaves e `for...of` percorre os valores. Num array `[10, 20, 30]`, usar `for...in` pode fazer a soma virar uma concatenação com os índices `"0"`, `"1"` e `"2"`.

O caso do Zune foi o exemplo mais marcante dessa parte. O laço deveria reduzir a quantidade de dias até chegar ao ano correto. Só que, quando o valor é exatamente 366 em um ano bissexto, a condição do `while` continua verdadeira e nenhum ramo muda a variável. Resultado: o laço fica preso. Eu entendi esse problema como uma falta de progresso. Toda repetição precisa caminhar para a condição de parada; se existe uma situação em que nada muda, o programa pode entrar em loop infinito.

Também aparece o `do-while`, que executa pelo menos uma vez porque testa a condição no final. Parece simples, mas é justamente o tipo de detalhe que dá erro quando eu esqueço como a estrutura funciona.

8. Goto, comandos protegidos e controle do fluxo

A parte do goto foi interessante porque o material não apresenta o comando só como uma coisa antiga. Ele usa um caso real para mostrar por que saltos livres podem ser perigosos. No exemplo simplificado do goto fail, existe uma linha goto fail repetida fora do if. Visualmente ela parece ligada à condição, mas como não há chaves, o salto acontece sempre. Com isso, uma verificação importante deixa de ser executada.

O que mais me chamou atenção nesse caso foi a combinação de vários detalhes pequenos: uma linha duplicada, falta de chaves e uma variável que ainda pode estar com valor de sucesso. Nenhum desses pontos parece gigantesco sozinho, mas juntos criam um problema sério. Pra mim, essa foi uma das melhores partes da aula porque mostra que bugs grandes podem nascer de erros bem pequenos de estrutura.

O material também comenta que break, continue e return podem ser vistos como desvios mais limitados. A diferença é que eles têm um objetivo bem definido e um alcance mais fácil de entender. Isso reforçou minha impressão de que o problema não é simplesmente 'desviar o fluxo', e sim ter liberdade demais sem uma estrutura clara.

Depois aparecem os comandos protegidos de Dijkstra. A ideia é ter várias guardas e escolher uma entre as que forem verdadeiras. Quando mais de uma condição pode ser usada, a escolha pode ser não determinística. No exemplo de ordenação, mesmo escolhendo guardas diferentes ao longo da execução, o processo chega ao resultado correto.

Go traz o select como uma ideia relacionada. Quando mais de um canal está pronto, um dos casos pode ser escolhido. Haskell aparece como contraste porque suas guardas são verificadas em ordem. Eu gostei dessa parte porque ela sai um pouco do if/else tradicional e mostra que existem outras formas de pensar o controle do programa. No geral, eu entendi que estruturas de controle são escolhas de projeto da linguagem e que cada escolha tenta equilibrar liberdade, clareza e segurança.

9. Exercícios: os bugs que eu encontrei

Os seis exercícios finais juntam praticamente tudo o que apareceu antes, e eu achei essa parte muito boa para revisar. No primeiro, em JavaScript, o código tenta somar [10, 20, 30] usando `for...in`. Só que ele percorre os índices e não os valores. Como os índices são Strings, o total acaba virando uma concatenação. A correção é usar `for...of` ou outra forma de percorrer os valores.

No segundo exercício, em Python, a função de desconto usa `d = d or 10`. A intenção é colocar 10 como padrão quando não existe desconto informado. O problema é que 0 também é falso, então um desconto de 0% é trocado por 10%. A solução correta é testar especificamente se `d is None`. Pra mim, esse exercício resume muito bem a ideia de truthiness.

No terceiro, o switch do Java não tem break. O dia recebe um valor válido, mas a execução continua nos casos seguintes e termina no default, deixando o nome como inválido. No quarto, em C, existe um ponto e vírgula logo depois do `if`. Esse ponto e vírgula vira uma instrução vazia e faz o `printf` ficar fora da condição. É o tipo de erro que quase some no meio do código.

No quinto exercício, em Go, aparece `x := x * 2` dentro do `if`. O `:=` cria outra variável `x` naquele bloco. Por isso, dentro do `if` aparece 20, mas fora continua 10. Para alterar a variável original, é preciso usar `=`. No sexto, em Java, `7 / 10` acontece como divisão inteira antes de chegar ao `double`, e o resultado da porcentagem vira 0.0%.

O que eu mais gostei nesses exercícios é que nenhum bug precisa de um programa enorme. Todos cabem em poucas linhas. Isso mostra que aprender programação não é só saber montar um algoritmo; eu também preciso conhecer a semântica da linguagem. Uma palavra, um operador, um ponto e vírgula ou um tipo diferente podem mudar completamente o resultado.

10. Autopesquisa, comparação final e o que eu aprendi

Na parte de autopesquisa, Ruby e Lisp aparecem para mostrar modelos diferentes dos exemplos mais tradicionais. Em Lisp, a notação prefixa elimina boa parte da dúvida de precedência. Em vez de escrever $a + b * c$, a expressão aparece como $(+ a (* b c))$. Os parênteses deixam a ordem explícita. Em Ruby, os operadores funcionam como métodos, então $1 + 2$ pode ser entendido como uma chamada parecida com $1.+(2)$. Isso ajuda a entender por que a linguagem permite redefinir operadores em classes próprias.

Depois de passar por tudo, eu comecei a enxergar um padrão entre as linguagens. C dá muita liberdade e deixa várias decisões nas mãos do programador. Java coloca algumas regras a mais, como booleano obrigatório no if e certas conversões controladas. Python facilita bastante a leitura e trabalha muito com iteráveis e truthiness. JavaScript é muito flexível com coerções, mas essa flexibilidade exige bastante cuidado. Go e Rust parecem ir mais para o lado de tornar decisões explícitas e barrar classes inteiras de erros antes da execução.

O principal que eu aprendi é que muitos bugs não acontecem porque a pessoa não sabe o que quer fazer. Às vezes a lógica geral está certa, mas uma regra pequena da linguagem muda o comportamento. Um switch sem break, uma divisão inteira, uma coerção automática, um laço que não progride ou uma variável sombreada já são suficientes para criar um resultado errado.

Os casos reais do Zune e do goto fail deixaram isso bem concreto para mim. Eles mostram que detalhes de estrutura e avaliação podem ter consequências grandes. Por isso, depois de explorar esse material, eu considero importante escrever de um jeito que deixe a intenção visível: usar parênteses quando a expressão pode confundir, usar chaves nos blocos, preferir comparações sem coerção, evitar efeitos colaterais escondidos e pensar se todo laço realmente caminha para terminar.

No fim, eu gostei da proposta exploratória porque prever a saída e depois conferir o resultado me ajudou a entender as regras de verdade. Assim, ficou mais fácil perceber as armadilhas de cada linguagem.